

Submission: Agentic Software Engineering System — URL Shortener

Assignment: Build an Agentic Software Engineering System (URL Shortener), evaluated primarily on the design and implementation of an agentic orchestration layer governing the SDLC, demonstrated across greenfield/brownfield/ambiguous scenarios.

What this document is: a self-contained submission write-up. Deeper detail on any section lives in the linked [docs/](#) files, but this document doesn't require reading them to follow the full story.

1. Objective and Approach

The assignment asked for two distinct things: a working URL-shortener prototype, and an agentic orchestration layer that governs how changes are made to it — explicitly named the "critical differentiator" and the top evaluation criterion.

The prototype was built first, in three phases (schema/CI scaffolding -> core shorten/redirect/expiry/click-count features -> hardening: custom aliases, retention policy, strict URL validation). That work is functionally complete and not the focus of this submission on its own — it's the substrate the orchestration layer operates on.

The core decision for the actual scope of this submission: build a **real, runnable orchestration engine** rather than documenting an orchestration *policy* and treating ad-hoc process as an implicit substitute. Only a real engine can demonstrate that a gate actually blocks progress, a retry actually re-executes a failed step, and a rollback actually restores prior state — which is exactly what "effectiveness of agentic orchestration" and "realism/quality of outputs" ask for.

Three scenarios were then run **through** that engine, each chosen to exercise a different core requirement:

- **Greenfield** — QR code generation. A brand-new capability, no existing code touched.
 - **Brownfield** — CORS configuration. An enhancement to *existing* code, chosen because it was already a documented gap in the repo, giving genuine before/after codebase reasoning to demonstrate.
 - **Ambiguous** — "add analytics." Deliberately vague, chosen to exercise requirement-understanding under ambiguity and a human-approval checkpoint placed on the *interpretation itself*, not just on release.
-

2. What Was Built

2.1 Backend (Java 21 / Spring Boot 3.3.5)

Feature	Notes
---------	-------

Shorten a URL	Normalizes + dedupes identical URLs; SHA-256 -> Base62 code generation with collision retry
Custom alias	Optional 6-12 char alias; silently falls back to generated code if taken
Redirect	302 with click-count increment + click-event logging
Expiry + retention	Configurable default expiry; scheduled purge job for old expired links
Strict URL validation	http/https only, rejects gibberish text and file-like hosts (.exe, .pdf, etc.)
CORS	Configurable, secure-by-default (no origins allowed until explicitly set)
QR code	GET /api/urls/{shortCode}/qr — PNG via ZXing
Analytics	GET /api/urls/{shortCode}/analytics — daily click counts + top referrers, aggregated in Java for testability

Stack: PostgreSQL + Flyway migrations, Hibernate (ddl-auto: validate), Testcontainers for integration tests, Docker + GitHub Actions CI/CD, JaCoCo coverage gate.

Full detail: [docs/BACKEND.md](#), [docs/schemas.md](#), [openapi.yml](#).

2.2 Orchestration engine (orchestrator/, Python, standard library only)

```
orchestrator/
pipeline.py Stage/Pipeline model; loads a JSON pipeline spec, validates the
dependency graph (unknown deps, cycles) at load time.
engine.py The scheduler: entry/exit gates, parallel+sequential execution,
bounded retry, git-backed rollback, safe-stop, human-approval
checkpoints, dynamic re-planning, cross-stage context.
audit.py Append-only JSON-lines audit log per run + human-readable summary.
metrics.py Success rate, retry/rollback frequency, MTTR, end-to-end latency
– per run and aggregate.
guardrails.py Policy checks against the real git diff: secrets, protected files,
change-size bound.
stages/ Generic stage runners shared by all 3 pipelines: requirements,
design, implementation, test, docs, compliance, release_readiness.
pipelines/*.json One spec per scenario.
artifacts/ Requirements/design write-ups + release manifests (tracked in git –
real authored content, not disposable logs).
runs/ Per-run audit logs (generated, gitignored).
tests/ 19 unit tests for the engine itself.
run.py CLI: start / approve / status / report.
```

Why Python, stdlib only: standalone from the Java build, zero install friction for evaluators (python orchestrator/run.py ..., no pip install).

Full architecture detail, including a real bug this design caught (an orchestrator audit file getting swept into a product commit and then deleted by a rollback — fixed by gitignoring `orchestrator/runs/`): [docs/ORCHESTRATION_ARCHITECTURE.md](#).

3. How the Orchestration Model Satisfies Each Requirement

Assignment requirement	How it's implemented
Explicit dependency graph with entry/exit gates	<code>pipeline.py</code> + <code>engine.py</code> — stages declare <code>depends_on</code> ; entry gate = deps satisfied; exit gate = stage-specific real check (test exit code, doc presence, compliance pass)
Sequential and parallel paths with synchronization	Stages with satisfied deps run concurrently (<code>ThreadPoolExecutor</code>); <code>test</code> and <code>docs</code> run in parallel in every pipeline, converging at compliance
Preserve cross-stage context and decision lineage	A single <code>RunContext</code> object threads through every stage; snapshotted to <code>state.json</code> after each one
Human approval checkpoints for high-impact actions	<code>requires_approval: true</code> halts the run until an explicit, audited <code>approve</code> command; used both at final release <i>and</i> , in the ambiguous scenario, on the requirements stage itself
Bounded retries, fallback, rollback, safe-stop	<code>max_retries</code> + backoff per stage; <code>rollback_on_failure</code> resets the repo to the stage's pre-execution git SHA; any unrecoverable failure safe-stops the whole run, marking downstream stages skipped
Policy guardrails for security/compliance/change control	<code>guardrails.py</code> : no-secrets-in-diff, no-unapproved-protected-files, change-size bound — real checks against the real diff
Audit-grade observability and traceability	Every stage transition is one JSON line in <code>audit.jsonl</code> : timestamp, stage, event, detail
Reliability metrics (success rate, retry/rollback frequency, MTTR, latency)	<code>metrics.py</code> , computed per-run and aggregate — see <code>orchestrator/artifacts/aggregate_metrics_report.json</code>
Dynamic re-planning when upstream outputs change	A stage can return <code>replan_to: "<earlier stage>"</code> ; engine reopens that stage and everything downstream, bounded to one replan per stage per run
Controlled autonomy	Mechanical stages (<code>test/docs/compliance</code>) are fully automated via real <code>subprocess</code> calls; cognitive stages (<code>requirements/design/implementation</code>) are produced by the AI agent and gated/logged by the engine exactly like mechanical stages — see Limitations (§6)

4. The Three Scenarios

Each was driven through the real engine against this repo — the audit logs referenced are real, not hand-written. Full detail: [docs/SCENARIOS.md](#).

Greenfield — QR Code Generation

Requirements/design artifacts normalized the request into acceptance criteria and a design choosing ZXing (confirmed before adding a new dependency, rather than hand-rolling QR encoding). Pipeline: `requirements -> design -> implementation -> parallel test+docs -> compliance -> approval-gated release_readiness`.

This run surfaced real bugs, not scripted ones: a `mvnw.cmd` path-resolution failure under Windows subprocess handling, a missing `JAVA_HOME` in the subprocess environment, and the `compliance` guardrail correctly blocking the new dependency's `pom.xml` change by default. Each was fixed for real (engine code fixes, committed immediately so a rollback couldn't discard them; a documented per-pipeline compliance override with a recorded rationale for the last one). Final run: **7/7 stages passed**, human-approved for release.

Brownfield — CORS Configuration

Requirements artifact explicitly enumerated impacted areas *before* touching anything: no existing `WebMvcConfigurer` (net-new, not a modification), a new config property needed, all controllers affected equally (cross-cutting, no controller changes needed), two docs files with now-stale warnings that needed correcting, not supplementing. Design: secure-by-default (empty allowed-origins list). Same pipeline shape as greenfield; completed cleanly on the first attempt (the bugs found during greenfield were already fixed). **7/7 stages passed**.

Ambiguous — "Add Analytics"

The key differentiator scenario. "Add analytics" doesn't specify what to measure, how it's surfaced, or who can see it. The `requirements` stage's own exit gate enforces this structurally: when a stage is flagged `ambiguous: true`, its artifact **must** contain an `## Assumptions` section or the stage fails regardless of approval. Seven explicit assumptions were documented (daily counts + top referrers only; no unique-visitor/geo/device tracking; a new dedicated endpoint; no new UI; same no-auth model; no historical backfill).

The pipeline places `requires_approval: true` on the **requirements stage itself**, not just release — the run genuinely halted twice: once immediately after `requirements` (before any design or implementation work), and again before release. This is the concrete demonstration of "human approval checkpoints for high-impact actions" applied to the action that matters most for an ambiguous input: signing off on *which interpretation is correct* before spending engineering effort building it. **7/7 stages passed, 2 human approvals recorded**.

Aggregate evidence

`orchestrator/artifacts/aggregate_metrics_report.json`: **3 runs, 21/21 stages passed, 100% success rate**. Zero retries/rollbacks in these *final* recorded runs — a sign of correct implementation by

the time each was finalized, not a gap in the mechanism: retry/rollback/safe-stop/replan are independently verified by 19 dedicated unit tests (including a real-git-repo rollback test), and real rollbacks *did* fire during iterative development of the greenfield scenario (see above).

5. Testing Approach

- **Backend:** TDD throughout. 80 unit/slice tests passing locally (Mockito unit tests, `@WebMvcTest` slice tests including real CORS preflight assertions). Testcontainers-based integration tests (`ShortUrlEndToEndTest`, `ShortUrlConcurrencyStressTest`, `UrlShortenerApplicationTests`) compile and are designed for CI; blocked locally by a Docker Desktop/Testcontainers version incompatibility on this dev machine, investigated and documented (not a code defect) in [docs/TESTING_AND_LIMITATIONS.md](#).
- **Orchestrator:** 19 `unittest` tests exercising real control-flow — a fixture stage that genuinely fails N times before passing (not a mock), a real throwaway git repo for the rollback test, a full approval pause/resume cycle, bounded re-planning.
- **Scenario-level system tests:** the three pipeline runs themselves, producing real audit logs inspected in `docs/SCENARIOS.md`.

Full breakdown: [docs/TESTING_AND_LIMITATIONS.md](#).

6. Limitations and Trade-offs (Honest Account)

- **No live LLM call inside the engine's cognitive stages.** requirements/design stages validate artifact shape; they don't autonomously generate the content. In this project, the AI agent authored those artifacts interactively and submitted them for the engine to gate — an accurate description of how this was actually built, not a shortcut hidden from view.
- **Single-machine execution.** Real concurrency (thread-pool batches), no distributed/multi-node coordination.
- **CLI-based approval, not a UI.** Real and auditable, but a production system would likely want a web UI or ChatOps integration.
- **Bounded re-planning depth fixed at 1** — guarantees termination, at the cost of a stage that would need two rounds of upstream rework instead safe-stopping.
- **Local Testcontainers execution is unreliable on this dev machine** — investigated in depth (see below), root-caused to Docker Desktop's own instability, not a version-compatibility bug that a `pom.xml` change can fix. CI is the reliable path.
- **`click_events` has no retention/purge policy yet** and analytics aggregation is done in-memory in Java (chosen deliberately for unit-testability over raw query efficiency) — both explicitly flagged as deferred, not overlooked, in `orchestrator/artifacts/ambiguous/design.md`.

Investigation note: Testcontainers/Docker Desktop

When asked to analyze `target/surefire-reports` and fix the 3 failing Testcontainers- dependent test classes, I attempted a real fix: bumped `testcontainers.version` (1.20.4 -> 1.20.6 -> 1.21.3) and tried `TESTCONTAINERS_RYUK_DISABLED=true`. Result: non-deterministic — identical commands produced three different failure points across successive runs. This points to Docker Desktop's own backend being flaky on this machine, not a fixable dependency-version issue. The version bump was reverted (no proven benefit, not worth leaving an unverified change in the deliverable); all 80 unaffected tests were re-verified green after the revert.

Full detail: [docs/TESTING_AND_LIMITATIONS.md](#).

7. Setup and Verification

```
# Backend
docker compose up -d
./mvnw spring-boot:run # http://localhost:8080
./mvnw test

# Orchestrator
python orchestrator/run.py start --pipeline orchestrator/pipelines/greenfield.json
--run-id my-run
python -m unittest discover -s orchestrator/tests -t .
python orchestrator/run.py report --all
```

Full instructions, including prerequisites and repo layout: [docs/SETUP.md](#).

8. Assumptions

- Evaluator has Python 3.9+ and a JDK 21 + Docker setup, consistent with what the backend already required before this work started.
 - "Add analytics" was interpreted per `orchestrator/artifacts/ambiguous/requirements.md` and explicitly approved in the real audit log before implementation — a different reviewer might reasonably interpret it differently, which is the point of the checkpoint existing.
 - No frontend is part of this submission — this repo contains the backend and the orchestration engine only. `docs/apiflow.md` and `docs/FRONTEND_INTEGRATION.md` are provided so a frontend can be built against this API, but none was built here.
 - All work is committed locally; nothing has been pushed to a remote unless separately requested, per this project's standing instruction to never push without explicit confirmation.
-

9. Repository Structure

```
src/ Spring Boot backend (Java 21)
docs/ All narrative documentation
```

openapi.yml Formal API contract
orchestrator/ Orchestration engine (Python, stdlib only)
pipelines/*.json Three scenario pipeline specs
artifacts/ Requirements/design write-ups + release manifests + aggregate metrics
runs/ Per-run audit logs (generated, gitignored – regenerate by re-running)
tests/ Engine's own 19-test unit suite
README.md Project overview and quick start
SUBMISSION.md This document

10. Document Index

Document	Contents
README.md	Project overview, quick start
docs/ORCHESTRATION_ARCHITECTURE.md	Full orchestration engine architecture
docs/SCENARIOS.md	Full three-scenario write-up with real evidence
docs/TESTING_AND_LIMITATIONS.md	Full testing approach, limitations, trade-offs
docs/FINAL_SUMMARY.md	Plan, rationale, artifacts, risks, assumptions
docs/SETUP.md	Setup and run instructions
docs/BACKEND.md	Backend feature reference
docs/schemas.md	Database and JSON schemas
docs/apiflow.md	Frontend implementation flows
docs/FRONTEND_INTEGRATION.md	Frontend integration gotchas
openapi.yml	Formal API contract